1. Removing a File from Git History
A common Git mistake is committing a large file or sensitive information such as a password, API key, or private configuration file. Simply deleting the file in a later commit does not remove it from the repository's history because the earlier commit still contains the file.To practice this, I would first add a test file, commit it, make another commit, and then remove the file from the repository history. For a repository where the file must be removed from every commit, a modern approach is to use git-filter-repo. For example:

```
git add test-secret.txt

git commit
-m "Add
test file"

git rm
test-secret.txt

git commit
-m "Remove test file"

git filter-repo --path
test-secret.txt
--invert-paths
```

The important difference is that deleting the file normally creates a new commit in which the file is gone,while rewriting the history removes the file from the affected commits. Because history rewriting changes commit IDs, it should be done carefully, especially on a shared repository. If the repository has already been pushed, the rewritten history generally needs to be force-pushed, and collaborators may need to re-clone or otherwise synchronize their repositories. If the removed file contained a password, token, or other secret, the secret should also be revoked or replaced; deleting it from Git history alone does not make an exposed credential safe.

2. git stash, git log --all --oneline, and git stash pop
I cloned a repository, modified one of its existing files, and then ran git stash. The working changes were temporarily saved by Git and removed from my working directory, so the file returned to the state of the most recent commit. This is useful when I have unfinished changes but need to temporarily switch branches, pull changes, or work on something else without committing the unfinished work. When running git log --all --oneline, the normal commit history is displayed for the reachable branches and references. A stash is stored by Git as a special reference, so depending on the repository and Git version, the stash commits can be included in the output when all references are shown. The stash itself is not a normal branch commit, but it can appear as a commit-like entry associated with refs/stash.

```
git stash
```

git log --all --oneline

git stash
pop
Finally, git stash pop reapplies the saved changes to the working directory and removes that stash entry if the operation succeeds. This is useful when I need to pause unfinished work temporarily, handle another task, and then return to exactly where I left off. If the stashed changes conflict with changes made while they were stashed, Git may report conflicts that must be resolved manually.Conclusion These exercises show that Git tracks project history rather than only the current files. They also demonstrate how git stash can temporarily store unfinished work without creating a permanent comm